🎓 Capstone Project - Advanced Machine Learning

TEC-VIII Programa de Especialización en Big Data Analytics aplicada a los Negocios

---

📋 Información del Proyecto

Campo	Información
Nombre del Estudiante	Javier Nuñez
Título del Proyecto	Deep Cost Control: Predicción de Gastos Operativos para Vehículos 4x4
Fecha de Entrega	[Completar]
Profesor	Carlos Mariño

📑 Índice

1. Resumen Ejecutivo
2. Configuración del Entorno
3. Definición del Problema de Negocio
4. Carga y Exploración de Datos
5. Preprocesamiento de Datos
6. Diseño y Arquitectura del Modelo
7. Entrenamiento del Modelo
8. Evaluación y Métricas
9. Interpretación de Resultados
10. Conclusiones y Recomendaciones de Negocio
11. Referencias

---

1. Resumen Ejecutivo

Instrucciones: Proporcione un resumen conciso (máximo 300 palabras) que incluya:

- Problema de negocio abordado
- Metodología utilizada
- Principales hallazgos
- Impacto esperado en el negocio

---

1. Problema de Negocio Abordado

La gestión de flotas en el sector minero e industrial enfrenta una alta variabilidad en los costos de mantenimiento, lo que dificulta la planificación financiera y pone en riesgo los márgenes de utilidad de NURA. Los métodos de estimación tradicionales (promedios simples o modelos lineales) no logran capturar las interdependencias complejas entre el uso, el tipo de reparación y la estacionalidad, generando incertidumbre en el flujo de caja operativo.

2. Metodología Utilizada

Se desarrolló e implementó un modelo de Deep Learning utilizando la librería PyTorch. La arquitectura consistió en una red neuronal profunda de tres capas, optimizada mediante técnicas de Batch Normalization y Dropout para asegurar la robustez y evitar el sobreajuste. El rendimiento se validó comparándolo con modelos baseline de Regresión Lineal y Random Forest. Adicionalmente, se aplicó la metodología SHAP para dotar al modelo de interpretabilidad, identificando los factores críticos que disparan el gasto.

3. Principales Hallazgos

- Superioridad del Modelo: La red neuronal superó a los modelos tradicionales con un R^2 de 0.5639, demostrando una mayor capacidad para explicar la varianza de los costos.
- Precisión Operativa: Se alcanzó un MAE (Error Medio Absoluto) de S/ 99.22, permitiendo presupuestar mantenimientos con un margen de error promedio menor a los 100 soles.
- Drivers de Costo: Se identificó que la gestión de llantas es el principal motor de gasto, seguido por el kilometraje y el volumen de repuestos solicitados.

4. Impacto Esperado

La implementación de esta herramienta permitirá a NURA optimizar su rentabilidad interna mediante una previsibilidad financiera superior. Al reducir la incertidumbre en los gastos de mantenimiento, la gerencia puede mejorar la negociación con proveedores estratégicos y asegurar que el escalamiento de la flota sea financieramente sostenible, protegiendo el margen neto ante cualquier expansión operativa.

---

2. Configuración del Entorno

2.1 Verificación de GPU (Recomendado para Deep Learning)

```
# Verificar si hay GPU disponible
import torch

# Verificar disponibilidad de GPU
if torch.cuda.is_available():
    print(f"✅ GPU disponible: {torch.cuda.get_device_name(0)}")
    print(f"   Memoria GPU: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")
    device = torch.device('cuda')
else:
    print("⚠️ GPU no disponible. Usando CPU.")
    print("   Recomendación: En Colab, vaya a Runtime > Change runtime type > GPU")
    device = torch.device('cpu')

print(f"\nDispositivo seleccionado: {device}")
```

```
✅ GPU disponible: Tesla T4
   Memoria GPU: 15.64 GB

Dispositivo seleccionado: cuda
```

2.2 Instalación de Librerías Adicionales (si es necesario)

```
# Descomente e instale las librerías adicionales que necesite
# !pip install transformers
# !pip install pytorch-lightning
# !pip install optuna
!pip install shap
# !pip install lime
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.51.0)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from shap) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from shap) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: llvmlite in /usr/local/lib/python3.12/dist-packages (from shap) (0.43.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.6.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)
```

2.3 Importación de Librerías

```
# ====================================================
# LIBRERÍAS FUNDAMENTALES
# ====================================================

# Manipulación de datos
import numpy as np
import pandas as pd

# Visualización
import matplotlib.pyplot as plt
import seaborn as sns

# Deep Learning - PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset, TensorDataset

# Deep Learning - TensorFlow/Keras (alternativa)
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, callbacks

# Preprocesamiento
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report,
                             mean_squared_error, mean_absolute_error, r2_score)

# Utilidades
import warnings
warnings.filterwarnings('ignore')
import shap
import joblib

# Configuración de visualización
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette('husl')
%matplotlib inline

# Semilla para reproducibilidad
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
torch.manual_seed(RANDOM_SEED)
tf.random.set_seed(RANDOM_SEED)

print("✅ Todas las librerías importadas correctamente")
print(f"   PyTorch version: {torch.__version__}")
print(f"   TensorFlow version: {tf.__version__}")
```

```
✅ Todas las librerías importadas correctamente
   PyTorch version: 2.10.0+cu128
   TensorFlow version: 2.19.0
```

2.4 Conexión con Google Drive (para cargar datos)

```
# Montar Google Drive para acceder a los datos
from google.colab import drive
drive.mount('/content/drive')

# Definir la ruta base de su proyecto
# Modifique esta ruta según la ubicación de sus datos
BASE_PATH = '/content/drive/MyDrive/Capstone_Project/'

print(f"✅ Google Drive montado")
print(f"   Ruta base del proyecto: {BASE_PATH}")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
✅ Google Drive montado
   Ruta base del proyecto: /content/drive/MyDrive/Capstone_Project/
```

---

3. Definición del Problema de Negocio

3.1 Contexto del Negocio

Instrucciones: Describa el contexto empresarial, incluyendo:

- Industria/Sector
- Empresa o caso de estudio
- Situación actual

NURA es una empresa dedicada a la gestión y alquiler de flotas de camionetas 4x4, operando principalmente en entornos exigentes como los sectores minero e industrial. La eficiencia operativa depende directamente de la disponibilidad de los vehículos, lo que implica un flujo constante de actividades de mantenimiento (reparaciones, repuestos y servicios mecánicos). Actualmente, se cuenta con una base de datos histórica robusta (2023-2026) que registra cada intervención realizada a las unidades.

3.2 Problema a Resolver

Instrucciones: Defina claramente:

- ¿Cuál es el problema específico?
- ¿Por qué es importante resolverlo?
- ¿Cuál es el impacto actual del problema?

- ¿Cuál es el problema específico? En NURA, la gestión de una flota de camionetas 4x4 destinadas a sectores críticos (minería e industria) implica un flujo constante de servicios de mantenimiento preventivo y correctivo. El problema central es la incapacidad de predecir con exactitud el costo económico de una intervención futura a partir de variables como el kilometraje acumulado, el tipo de servicio y el proveedor. Actualmente, el cálculo de presupuestos se realiza de forma manual y reactiva, careciendo de un modelo analítico que aprenda de la variabilidad histórica de los últimos tres años (2023-2026).
- ¿Por qué es importante resolverlo? Resolver esta brecha mediante Deep Learning es fundamental por tres razones:
  - Precisión Financiera: Permite transitar de una gestión basada en la "intuición" a una basada en datos, asegurando que las provisiones de capital para mantenimiento sean exactas.
  - Poder de Negociación: Al conocer el costo esperado de una reparación según el modelo, la gerencia puede validar cotizaciones de proveedores y evitar sobrecostos injustificados.
  - Disponibilidad Mecánica: Una mejor previsión de costos facilita la programación de mantenimientos complejos en camionetas con alto kilometraje antes de que ocurra una falla crítica que detenga la operación.
- ¿Cuál es el impacto actual del problema? La falta de una herramienta predictiva genera actualmente los siguientes efectos negativos:
  - Desviaciones Presupuestarias: El gasto real de mantenimiento excede frecuentemente las proyecciones iniciales, afectando el flujo de caja operativo.
  - Erosión de Márgenes: Los costos inesperados en repuestos y mano de obra reducen la rentabilidad neta por unidad arrendada.
  - Opacidad Operativa: Dificultad para identificar patrones de desgaste o anomalías en la facturación de servicios, lo que limita la capacidad de mejora continua en la gestión de la flota.

3.3 Objetivos del Proyecto

Instrucciones: Liste los objetivos SMART (Específicos, Medibles, Alcanzables, Relevantes, Temporales)

Objetivo General: Desarrollar un modelo de Deep Learning que prediga el costo total (INC IGV) de un mantenimiento de flota basándose en el historial de uso y tipo de servicio.

Objetivos Específicos:

1. S (Específico): Implementar una red neuronal para predecir costos operativos de vehículos 4x4.
2. M (Medible): Lograr un error absoluto medio (MAE) que sea aceptable para la toma de decisiones financieras (objetivo: $< 15\%$ de error).
3. A (Alcanzable): Entrenar el modelo con el dataset de más de 3,700 registros de mantenimiento de NURA.
4. R (Relevante): Optimizar la planificación financiera y el control de costos de la empresa.
5. T (Tiempo): Entregar el modelo funcional y el reporte final para finales de marzo de 2026.

3.4 Tipo de Problema de Machine Learning

Instrucciones: Identifique el tipo de problema:

- [ ] Clasificación binaria
- [ ] Clasificación multiclase
- [x] Regresión
- [ ] Clustering
- [ ] Series temporales
- [ ] Procesamiento de Lenguaje Natural (NLP)
- [ ] Visión por Computadora
- [ ] Otro: _____

Justificación: *El proyecto busca predecir el costo total de mantenimiento (una variable numérica continua en soles). No estamos intentando categorizar el mantenimiento, sino estimar un valor monetario preciso, lo que define técnicamente un problema de regresión.*

4. Carga y Exploración de Datos

4.1 Carga de Datos

```
# ====================================================
# CARGA DE DATOS
# ====================================================

# Opción 1: Cargar desde Google Drive
nombre_archivo = 'Trabajos Realizados 2023 -2025 ORIGINAL.xlsx'
nombre_hoja = '2023'
df = pd.read_excel(BASE_PATH + nombre_archivo, sheet_name=nombre_hoja)

# Opción 2: Cargar desde URL
# df = pd.read_csv('https://url-de-sus-datos.com/datos.csv')

# Opción 3: Cargar desde archivo local (subido a Colab)
# from google.colab import files
# uploaded = files.upload()
# df = pd.read_csv('nombre_archivo.csv')

# Opción 4: Dataset de ejemplo (para testing)
# from sklearn.datasets import load_iris, load_boston, fetch_california_housing
# data = load_iris()
# df = pd.DataFrame(data.data, columns=data.feature_names)
# df['target'] = data.target
```

```python
# ====================================================
# COMPLETE AQUÍ: Cargue su dataset
# ====================================================

# df = pd.read_csv('...')  # Descomente y complete

# Limpieza rápida
df.columns = df.columns.str.strip()
df['INC IGV'] = pd.to_numeric(df['INC IGV'], errors='coerce')
df['Kilometraje'] = pd.to_numeric(df['Kilometraje'], errors='coerce')
df = df.dropna(subset=['INC IGV', 'Kilometraje'])

# Quita las filas vacías
df = df.dropna(subset=['INC IGV', 'Kilometraje'])

print(f"✅ Dataset cargado exitosamente")
print(f"   Dimensiones: {df.shape[0]:,} filas × {df.shape[1]} columnas")
```

```
✅ Dataset cargado exitosamente
   Dimensiones: 3,194 filas × 13 columnas
```

```python
# ====================================================
# VISUALIZACIÓN DE ESTRUCTURA
# ====================================================

print("--- 1. TIPOS DE DATOS Y MEMORIA ---")
print(df.info())

print("\n--- 2. RESUMEN ESTADÍSTICO (Costos y Kilómetros) ---")
display(df[['Kilometraje', 'INC IGV']].describe())

print("\n--- 3. CONTEO DE VALORES NULOS ---")
print(df.isnull().sum())
```

```
--- 1. TIPOS DE DATOS Y MEMORIA ---
<class 'pandas.core.frame.DataFrame'>
Index: 3194 entries, 0 to 3540
Data columns (total 13 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   |              3194 non-null   object
 1   Mes            3194 non-null   float64
 2   Fecha          3194 non-null   datetime64[ns]
 3   Placa          3194 non-null   object
 4   Kilometraje    3194 non-null   float64
 5   Proveedor      3194 non-null   object
 6   #Factura       3183 non-null   object
 7   Descripcion    3187 non-null   object
 8   Q              3193 non-null   float64
 9   Tipo           3194 non-null   object
 10  Clasificacion  3171 non-null   object
 11  SIN IGV        3186 non-null   float64
 12  INC IGV        3194 non-null   float64
dtypes: datetime64[ns](1), float64(5), object(7)
memory usage: 349.3+ KB
None

--- 2. RESUMEN ESTADÍSTICO (Costos y Kilómetros) ---
```

	Kilometraje	INC IGV
count	3194.000000	3194.000000
mean	72483.643707	169.723963
std	36496.563472	283.647454
min	35.000000	0.000000
25%	44520.000000	50.000000
50%	74160.000000	106.200000
75%	99640.000000	185.274750
max	177079.000000	3150.498520

```
--- 3. CONTEO DE VALORES NULOS ---
|                 0
Mes               0
Fecha             0
Placa             0
Kilometraje       0
Proveedor         0
#Factura         11
Descripcion       7
Q                 1
Tipo              0
Clasificacion    23
SIN IGV           8
INC IGV           0
dtype: int64
```

4.2 Descripción del Dataset

Instrucciones: Describa su dataset:

- Fuente de los datos
- Período de tiempo que cubren
- Descripción de cada variable

Variable	Tipo	Descripción
"|"	object	Carácter de separación del archivo original.
Mes	float64	Mes calendario en que se realizó el gasto.
Fecha	datetime64	Fecha exacta del servicio (útil para series temporales).
Placa	object	Identificador de la unidad (permite ver el historial del vehículo).
Kilometraje	float64	Predictor clave: Desgaste acumulado de la camioneta.
Proveedor	object	Empresa que realizó el mantenimiento.
#Factura	object	Número de comprobante (no influye en el costo futuro).
Descripcion	object	Detalle textual del trabajo o repuesto comprado.
Q	float64	Cantidad de unidades o servicios realizados.
Tipo	object	Clasifica si es Mano de Obra, Repuesto o Insumo.
Clasificacion	object	Categoría técnica (Frenos, Suspensión, Motor, etc.).
SIN IGV	float64	Monto base del servicio (altamente correlacionado al total).

Variable	Tipo	Descripción
INC IGV	float64	El costo total que buscamos predecir.
target	float64	INC IGV: El costo total que el modelo debe predecir.

4.3 Exploración Inicial de Datos (EDA)

```
# ====================================================
# INFORMACIÓN GENERAL DEL DATASET
# ====================================================

print("=" * 60)
print("INFORMACIÓN GENERAL DEL DATASET")
print("=" * 60)

# Primeras filas
print("\n📊 Primeras 5 filas:")
display(df.head())

# Información del dataset
print("\n📋 Información del Dataset:")
print(df.info())

# Estadísticas descriptivas
print("\n📈 Estadísticas Descriptivas:")
display(df.describe())
```

```
============================================================
INFORMACIÓN GENERAL DEL DATASET
============================================================

📊 Primeras 5 filas:
```

	|	Mes	Fecha	Placa	Kilometraje	Proveedor	#Factura	Descripcion	Q	Tipo	Clasificacion	SIN IGV	INC IGV
0	.	1.0	2023-01-02	BKS-799	41207.0	AUTO BOUTIQUE	001-4037	INSTALACION DE PATIN SOPORTE	1.0	MANO DE OBRA	EQUIPAMIENTO	211.86	249.9948
1	2023	1.0	2023-01-02	BKS-799	41207.0	AUTOESPAR	009-76581	DIFF OIL 80W GL5	5.0	REPUESTOS	MANTENIMIENTO	107.32	126.6376
2	2023	1.0	2023-01-02	BKS-799	41207.0	AUTOESPAR	009-76581	TRANSFER OIL	1.0	REPUESTOS	MANTENIMIENTO	189.04	223.0672
3	2023	1.0	2023-01-02	BKS-799	41207.0	AUTOESPAR	009-76581	EMPAQUETADURA (TAPON DE DRENAJE DE DEOLD	1.0	REPUESTOS	MANTENIMIENTO	4.36	5.1448
4	2023	1.0	2023-01-02	BKS-799	41207.0	AUTOESPAR	009-76581	SUBCOJUNTO ELEMENTO FILTRO	1.0	REPUESTOS	MANTENIMIENTO	111.75	131.8650

```
📋 Información del Dataset:
<class 'pandas.core.frame.DataFrame'>
Index: 3194 entries, 0 to 3540
Data columns (total 13 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   |              3194 non-null   object
 1   Mes            3194 non-null   float64
 2   Fecha          3194 non-null   datetime64[ns]
 3   Placa          3194 non-null   object
 4   Kilometraje    3194 non-null   float64
 5   Proveedor      3194 non-null   object
 6   #Factura       3183 non-null   object
 7   Descripcion    3187 non-null   object
 8   Q              3193 non-null   float64
 9   Tipo           3194 non-null   object
 10  Clasificacion  3171 non-null   object
 11  SIN IGV        3186 non-null   float64
 12  INC IGV        3194 non-null   float64
dtypes: datetime64[ns](1), float64(5), object(7)
memory usage: 349.3+ KB
None

📈 Estadísticas Descriptivas:
```

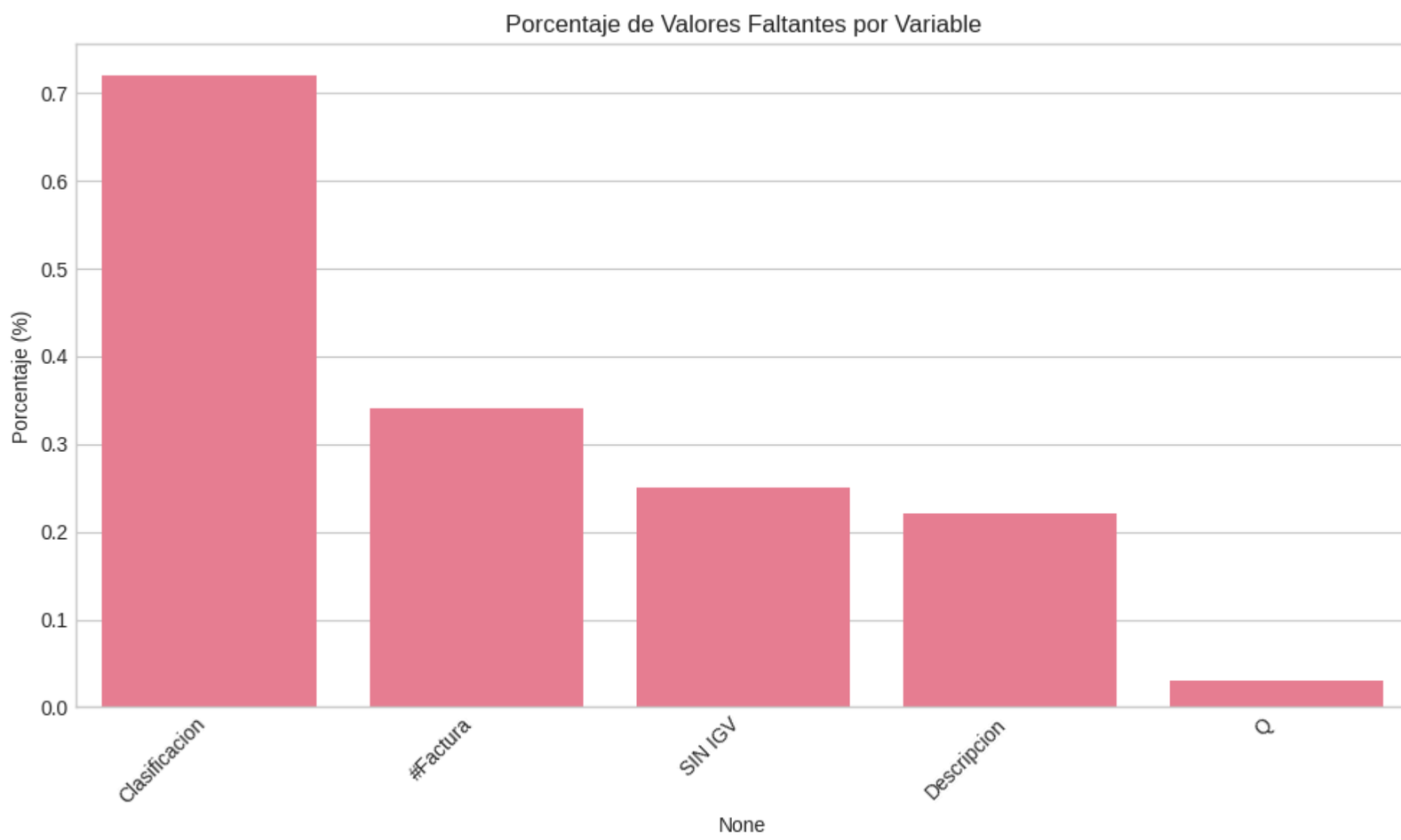
	Mes	Fecha	Kilometraje	Q	SIN IGV	INC IGV
count	3194.000000	3194	3194.000000	3193.000000	3186.000000	3194.000000
mean	5.528491	2024-04-16 17:55:15.967439104	72483.643707	1.766677	144.195033	169.723963
min	1.000000	2023-01-02 00:00:00	35.000000	1.000000	0.000000	0.000000
25%	2.000000	2023-05-09 00:00:00	44520.000000	1.000000	42.372911	50.000000
50%	5.000000	2024-01-26 00:00:00	74160.000000	1.000000	90.000000	106.200000
75%	9.000000	2025-03-17 00:00:00	99640.000000	1.000000	157.120000	185.274750
max	12.000000	2026-03-06 00:00:00	177079.000000	80.000000	2669.914000	3150.498520
std	3.595588	NaN	36496.563472	3.715913	240.572650	283.647454

```
# ====================================================
# ANÁLISIS DE VALORES FALTANTES
# ====================================================

print("=" * 60)
print("ANÁLISIS DE VALORES FALTANTES")
print("=" * 60)

# Calcular valores faltantes
missing_data = pd.DataFrame({
    'Total Faltantes': df.isnull().sum(),
    'Porcentaje (%)': (df.isnull().sum() / len(df) * 100).round(2)
})
missing_data = missing_data[missing_data['Total Faltantes'] > 0].sort_values('Porcentaje (%)', ascending=False)

if len(missing_data) > 0:
    print("\n⚠️ Variables con valores faltantes:")
    display(missing_data)

    # Visualización de valores faltantes
    plt.figure(figsize=(10, 6))
    sns.barplot(x=missing_data.index, y='Porcentaje (%)', data=missing_data)
    plt.title('Porcentaje de Valores Faltantes por Variable')
```

```
    plt.xticks(rotation=45, ha='right')
    plt.ylabel('Porcentaje (%)')
    plt.tight_layout()
    plt.show()
else:
    print("\n✅ No hay valores faltantes en el dataset")
```

```
============================================================
ANÁLISIS DE VALORES FALTANTES
============================================================

⚠️ Variables con valores faltantes:
```

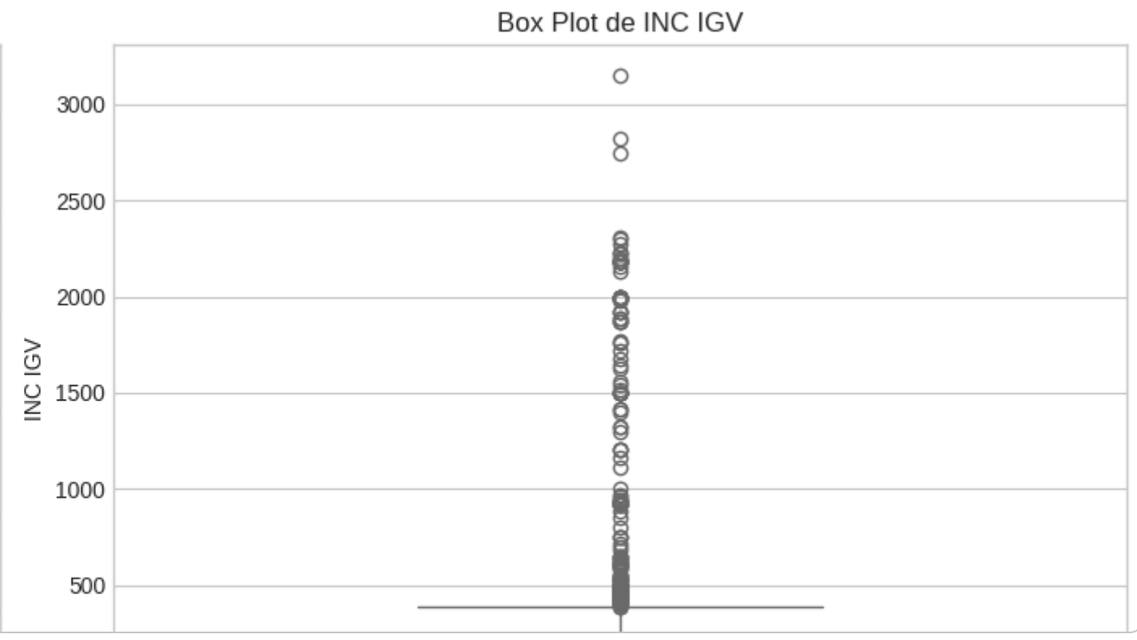
	Total Faltantes	Porcentaje (%)
Clasificacion	23	0.72
#Factura	11	0.34
SIN IGV	8	0.25
Descripcion	7	0.22
Q	1	0.03



Next steps: Generate code with `missing_data` New interactive sheet

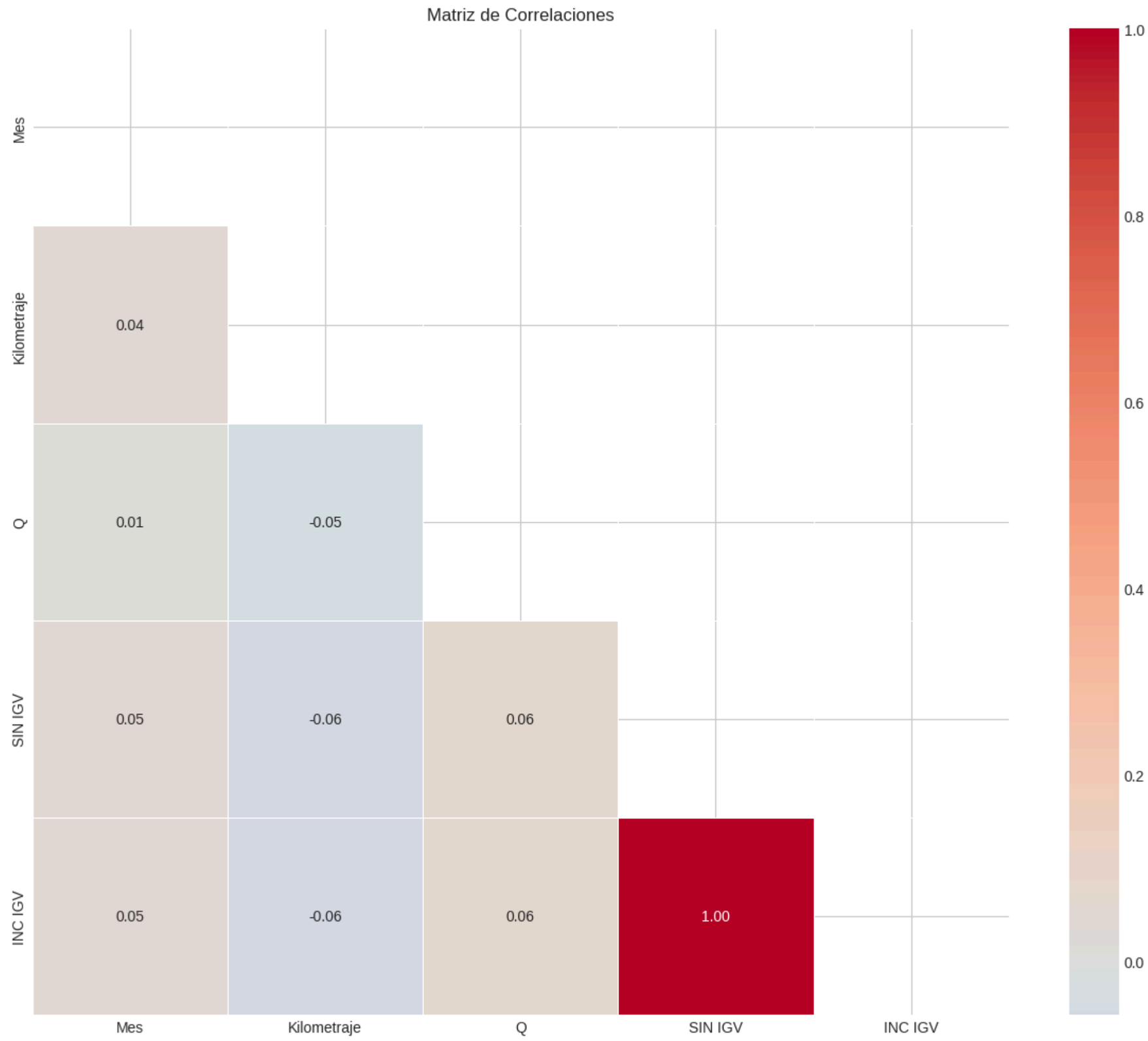
```
# ====================================================
# ANÁLISIS DE LA VARIABLE OBJETIVO
# ====================================================

# COMPLETE: Especifique el nombre de su variable objetivo
TARGET_COLUMN = 'INC IGV'  # Cambie 'target' por el nombre de su variable objetivo

print("=" * 60)
print(f"ANÁLISIS DE LA VARIABLE OBJETIVO: {TARGET_COLUMN}")
print("=" * 60)

# Para clasificación
if df[TARGET_COLUMN].dtype == 'object' or df[TARGET_COLUMN].nunique() < 20:
    print("\n📊 Distribución de clases:")
    class_dist = df[TARGET_COLUMN].value_counts()
    print(class_dist)

    # Visualización
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Gráfico de barras
    sns.countplot(data=df, x=TARGET_COLUMN, ax=axes[0])
    axes[0].set_title(f'Distribución de {TARGET_COLUMN}')
    axes[0].set_xlabel(TARGET_COLUMN)
    axes[0].set_ylabel('Frecuencia')

    # Gráfico de pastel
    axes[1].pie(class_dist.values, labels=class_dist.index, autopct='%1.1f%%', startangle=90)
    axes[1].set_title(f'Proporción de {TARGET_COLUMN}')

    plt.tight_layout()
    plt.show()

    # Verificar desbalance
    imbalance_ratio = class_dist.max() / class_dist.min()
    if imbalance_ratio > 3:
        print(f"\n⚠️ ADVERTENCIA: Dataset desbalanceado (ratio {imbalance_ratio:.2f}:1)")
        print("   Considere técnicas de balanceo: SMOTE, undersampling, class weights")
else:
    # Para regresión
    print("\n📊 Estadísticas de la variable objetivo:")
    print(df[TARGET_COLUMN].describe())

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Histograma
    sns.histplot(df[TARGET_COLUMN], kde=True, ax=axes[0])
    axes[0].set_title(f'Distribución de {TARGET_COLUMN}')

    # Box plot
    sns.boxplot(y=df[TARGET_COLUMN], ax=axes[1])
    axes[1].set_title(f'Box Plot de {TARGET_COLUMN}')

    plt.tight_layout()
    plt.show()
```

```
============================================================
ANÁLISIS DE LA VARIABLE OBJETIVO: INC IGV
============================================================

📊 Estadísticas de la variable objetivo:
count    3194.000000
mean      169.723963
std       283.647454
min         0.000000
25%        50.000000
50%       106.200000
75%       185.274750
max      3150.498520
Name: INC IGV, dtype: float64
```




```
# ====================================================
# ANÁLISIS DE CORRELACIONES
# ====================================================

print("=" * 60)
print("MATRIZ DE CORRELACIONES")
print("=" * 60)

# Seleccionar solo columnas numéricas
numeric_cols = df.select_dtypes(include=[np.number]).columns

if len(numeric_cols) > 1:
    # Calcular correlaciones
    correlation_matrix = df[numeric_cols].corr()

    # Visualización
    plt.figure(figsize=(12, 10))
    mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))
    sns.heatmap(correlation_matrix, mask=mask, annot=True, cmap='coolwarm',
                center=0, fmt='.2f', linewidths=0.5)
    plt.title('Matriz de Correlaciones')
    plt.tight_layout()
    plt.show()

    # Correlaciones con la variable objetivo
    if TARGET_COLUMN in numeric_cols:
        print(f"\n📊 Correlaciones con {TARGET_COLUMN}:")
        target_corr = correlation_matrix[TARGET_COLUMN].drop(TARGET_COLUMN).sort_values(ascending=False)
        print(target_corr)
else:
    print("⚠️ No hay suficientes columnas numéricas para análisis de correlación")
```

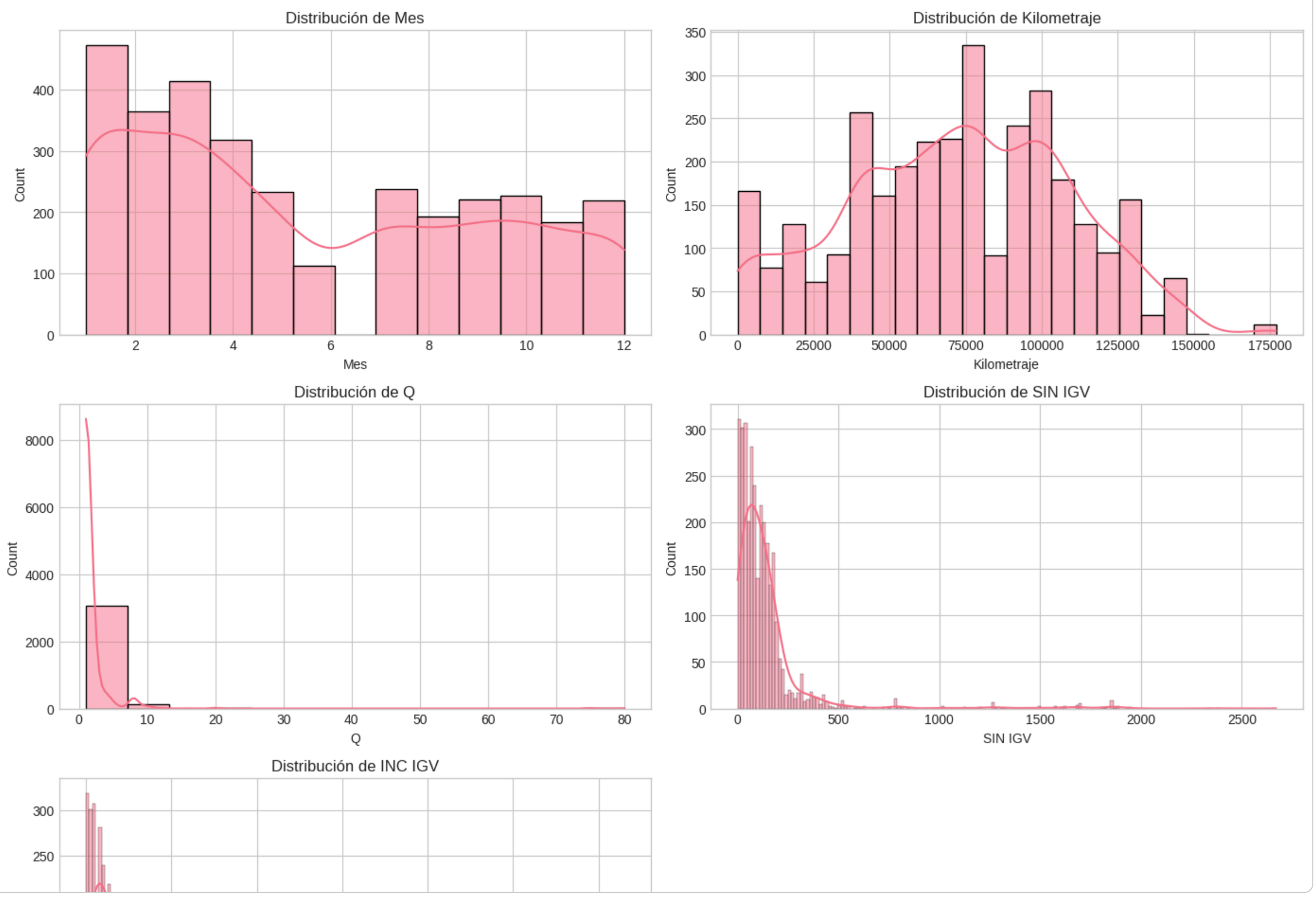
```
============================================================
MATRIZ DE CORRELACIONES
============================================================
```




```
📊 Correlaciones con INC IGV:
SIN IGV        1.000000
Q              0.064953
Mes            0.048846
Kilometraje   -0.057077
Name: INC IGV, dtype: float64
```

```
# ====================================================
# VISUALIZACIONES ADICIONALES
# ====================================================

print("=" * 60)
print("VISUALIZACIONES ADICIONALES")
print("=" * 60)

# Distribución de variables numéricas
numeric_cols_plot = df.select_dtypes(include=[np.number]).columns[:8]  # Primeras 8 columnas

if len(numeric_cols_plot) > 0:
    n_cols = 2
    n_rows = (len(numeric_cols_plot) + 1) // 2

    fig, axes = plt.subplots(n_rows, n_cols, figsize=(14, 4*n_rows))
    axes = axes.flatten() if n_rows > 1 else [axes]

    for i, col in enumerate(numeric_cols_plot):
        if i < len(axes):
            sns.histplot(df[col], kde=True, ax=axes[i])
            axes[i].set_title(f'Distribución de {col}')

    # Ocultar ejes vacíos
    for j in range(i+1, len(axes)):
        axes[j].set_visible(False)

    plt.tight_layout()
    plt.show()
```

```
============================================================
VISUALIZACIONES ADICIONALES
============================================================
```



4.4 Hallazgos del EDA

Instrucciones: Resuma los principales hallazgos de la exploración de datos:

Hallazgos Principales:

1. Relación Perfecta (Data Leakage): La matriz de correlación muestra un valor de 1.00 entre SIN IGV e INC IGV. Esto significa que una columna es prácticamente copia de la otra (más el impuesto), lo cual le daría la respuesta servida a la IA sin que esta aprenda realmente.
2. Independencia del Kilometraje: Se observa una correlación casi nula (-0.06) entre el Kilometraje y el costo. Esto indica que el precio de los servicios no sube de forma lineal con el recorrido, sino que depende de otros factores más complejos.
3. Sesgo de Costos: El histograma de INC IGV muestra que la gran mayoría de los gastos son menores a S/ 200 (mantenimientos preventivos), pero existe una "cola" de datos que llega hasta los S/ 3,150, que son los casos críticos.

Problemas Identificados:

1. Valores Atípicos (Outliers): En el gráfico de Q (cantidad) aparece un valor extremo de 80 unidades, mientras que el 99% de los datos es 1 o 2. También hay costos máximos muy alejados del promedio que podrían "marear" a la red neuronal.
2. *[Completar]*

Acciones a Tomar:

1. Datos Faltantes en Categorías: El análisis de nulos detectó 23 filas sin la variable Clasificacion. Al ser una de las "pistas" más importantes para entender el tipo de arreglo, no podemos dejar esos huecos.
2. Limpieza y Filtrado: Se eliminarán las filas con valores nulos en Clasificacion y se tratarán los valores extremos (outliers) para que el modelo aprenda de la operación normal y no de errores de registro o compras masivas excepcionales.

---

5. Preprocesamiento de Datos

5.1 Tratamiento de Valores Faltantes

```
# ====================================================
# TRATAMIENTO DE VALORES FALTANTES
# ====================================================

print("=" * 60)
print("TRATAMIENTO DE VALORES FALTANTES")
print("=" * 60)

# Crear copia del dataframe
df_clean = df.copy()

# Opción 1: Eliminar filas con valores faltantes
df_clean = df_clean.dropna()

# Opción 2: Imputar con la media (variables numéricas)
# from sklearn.impute import SimpleImputer
# imputer = SimpleImputer(strategy='mean')
# df_clean[numeric_cols] = imputer.fit_transform(df_clean[numeric_cols])

# Opción 3: Imputar con la moda (variables categóricas)
# for col in categorical_cols:
```

```
#     df_clean[col].fillna(df_clean[col].mode()[0], inplace=True)

# Opción 4: Imputación avanzada con KNN
# from sklearn.impute import KNNImputer
# imputer = KNNImputer(n_neighbors=5)
# df_clean[numeric_cols] = imputer.fit_transform(df_clean[numeric_cols])

# ====================================================
# COMPLETE AQUÍ: Aplique su estrategia de imputación
# ====================================================



print(f"\n✅ Valores faltantes tratados")
print(f"   Filas restantes: {len(df_clean):,}")
```

```
============================================================
TRATAMIENTO DE VALORES FALTANTES
============================================================

✅ Valores faltantes tratados
   Filas restantes: 3,151
```

5.2 Tratamiento de Outliers

```
# ====================================================
# DETECCIÓN Y TRATAMIENTO DE OUTLIERS
# ====================================================

print("=" * 60)
print("DETECCIÓN DE OUTLIERS")
print("=" * 60)

def detect_outliers_iqr(data, column):
    """Detecta outliers usando el método IQR"""
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]
    return len(outliers), lower_bound, upper_bound

# Detectar outliers en cada columna numérica
numeric_cols = df_clean.select_dtypes(include=[np.number]).columns

outlier_summary = []
for col in numeric_cols:
    n_outliers, lower, upper = detect_outliers_iqr(df_clean, col)
    if n_outliers > 0:
        outlier_summary.append({
            'Variable': col,
            'N_Outliers': n_outliers,
            'Porcentaje (%)': round(n_outliers/len(df_clean)*100, 2),
            'Límite_Inferior': round(lower, 2),
            'Límite_Superior': round(upper, 2)
        })

if outlier_summary:
    outlier_df = pd.DataFrame(outlier_summary)
    print("\n⚠️ Variables con outliers detectados:")
    display(outlier_df)
else:
    print("\n✅ No se detectaron outliers significativos")
```

```
============================================================
DETECCIÓN DE OUTLIERS
============================================================

⚠️ Variables con outliers detectados:
```

	Variable	N_Outliers	Porcentaje (%)	Límite_Inferior	Límite_Superior
0	Q	715	22.69	1.00	1.00
1	SIN IGV	200	6.35	-129.27	328.46
2	INC IGV	200	6.35	-152.54	387.58

Next steps: Generate code with `outlier_df` New interactive sheet

```
# ====================================================
# TRATAMIENTO DE OUTLIERS (OPCIONAL)
# ====================================================

# Opción 1: Eliminar outliers
# for col in numeric_cols:
#     Q1, Q3 = df_clean[col].quantile([0.25, 0.75])
#     IQR = Q3 - Q1
#     df_clean = df_clean[(df_clean[col] >= Q1 - 1.5*IQR) & (df_clean[col] <= Q3 + 1.5*IQR)]

# Opción 2: Capear outliers (winsorizing)
# from scipy.stats import mstats
# for col in numeric_cols:
#     df_clean[col] = mstats.winsorize(df_clean[col], limits=[0.05, 0.05])

# Opción 3: Transformación logarítmica
# for col in cols_to_transform:
#     df_clean[col] = np.log1p(df_clean[col])

# ====================================================
# COMPLETE AQUÍ: Aplique su estrategia de tratamiento
# ====================================================
```

5.3 Codificación de Variables Categóricas

```
# ====================================================
# CODIFICACIÓN DE VARIABLES CATEGÓRICAS
```

```
# ====================================================

print("=" * 60)
print("CODIFICACIÓN DE VARIABLES CATEGÓRICAS")
print("=" * 60)

# Identificar variables categóricas
categorical_cols = df_clean.select_dtypes(include=['object', 'category']).columns.tolist()
print(f"\nVariables categóricas encontradas: {categorical_cols}")

# Opción 1: Label Encoding (para variables ordinales o target)
# le = LabelEncoder()
# df_clean['columna_encoded'] = le.fit_transform(df_clean['columna'])

# Opción 2: One-Hot Encoding (para variables nominales)
# df_clean = pd.get_dummies(df_clean, columns=categorical_cols, drop_first=True)

# Opción 3: Target Encoding
# from sklearn.preprocessing import TargetEncoder
# encoder = TargetEncoder()
# df_clean[categorical_cols] = encoder.fit_transform(df_clean[categorical_cols], df_clean[TARGET_COLUMN])

# ====================================================
# COMPLETE AQUÍ: Aplique su estrategia de codificación
# ====================================================
# 1. Quitamos las columnas que son texto largo o IDs
cols_a_eliminar = ['Descripcion', 'Placa', 'Proveedor', '#Factura', '|', 'SIN IGV', 'Fecha']
df_clean = df_clean.drop(columns=[c for c in cols_a_eliminar if c in df_clean.columns])

# 2. Actualizamos la lista de variables que quedan para traducir
categorical_cols = df_clean.select_dtypes(include=['object', 'category']).columns.tolist()

# 3. Aplicamos la Opción 2 del profesor (One-Hot Encoding)
df_clean = pd.get_dummies(df_clean, columns=categorical_cols, drop_first=True)


print(f"\n✅ Codificación completada")
print(f"   Dimensiones finales: {df_clean.shape}")
```

```
============================================================
CODIFICACIÓN DE VARIABLES CATEGÓRICAS
============================================================

Variables categóricas encontradas: ['|', 'Placa', 'Proveedor', '#Factura', 'Descripcion', 'Tipo', 'Clasificacion']

✅ Codificación completada
   Dimensiones finales: (3151, 19)
```

5.4 Escalado/Normalización de Features

```
# ====================================================
# ESCALADO DE FEATURES
# ====================================================

print("=" * 60)
print("ESCALADO DE FEATURES")
print("=" * 60)

# Separar features y target
X = df_clean.drop(columns=[TARGET_COLUMN])
y = df_clean[TARGET_COLUMN]

print(f"\nDimensiones de X: {X.shape}")
print(f"Dimensiones de y: {y.shape}")

# Opción 1: StandardScaler (media=0, std=1) - Recomendado para redes neuronales
scaler = StandardScaler()

# Opción 2: MinMaxScaler (rango [0,1])
# scaler = MinMaxScaler()

# Opción 3: RobustScaler (robusto a outliers)
# from sklearn.preprocessing import RobustScaler
# scaler = RobustScaler()

# Aplicar escalado
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=X.columns, index=X.index)

print(f"\n✅ Escalado completado usando {type(scaler).__name__}")
print(f"   Media de features: {X_scaled.mean().mean():.6f}")
print(f"   Std de features: {X_scaled.std().mean():.6f}")
```

```
============================================================
ESCALADO DE FEATURES
============================================================

Dimensiones de X: (3151, 18)
Dimensiones de y: (3151,)

✅ Escalado completado usando StandardScaler
   Media de features: -0.000000
   Std de features: 1.000159
```

5.5 División de Datos (Train/Validation/Test)

```
# ====================================================
# DIVISIÓN DE DATOS
# ====================================================

print("=" * 60)
print("DIVISIÓN DE DATOS")
print("=" * 60)

# División en train (70%), validation (15%), test (15%)
X_temp, X_test, y_temp, y_test = train_test_split(
    X_scaled, y, test_size=0.15, random_state=RANDOM_SEED, stratify=y if y.dtype == 'object' or y.nunique() < 20 else None
)

X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.176, random_state=RANDOM_SEED, stratify=y_temp if y_temp.dtype == 'object' or y_temp.nunique() < 20 else None  # 0.176 ≈ 15% del to
)
```

```
print(f"\n📊 División de datos:")
print(f"   Training set:   {X_train.shape[0]:,} muestras ({X_train.shape[0]/len(X_scaled)*100:.1f}%)")
print(f"   Validation set: {X_val.shape[0]:,} muestras ({X_val.shape[0]/len(X_scaled)*100:.1f}%)")
print(f"   Test set:       {X_test.shape[0]:,} muestras ({X_test.shape[0]/len(X_scaled)*100:.1f}%)")

# Verificar distribución de clases (para clasificación)
if y.dtype == 'object' or y.nunique() < 20:
    print(f"\n📊 Distribución de clases en cada conjunto:")
    print(f"   Train: {dict(y_train.value_counts(normalize=True).round(3))}")
    print(f"   Val:   {dict(y_val.value_counts(normalize=True).round(3))}")
    print(f"   Test:  {dict(y_test.value_counts(normalize=True).round(3))}")
```

```
============================================================
DIVISIÓN DE DATOS
============================================================

📊 División de datos:
   Training set:   2,206 muestras (70.0%)
   Validation set: 472 muestras (15.0%)
   Test set:       473 muestras (15.0%)
```

5.6 Preparación de Datos para Deep Learning

```
# ====================================================
# PREPARACIÓN PARA PYTORCH
# ====================================================

print("=" * 60)
print("PREPARACIÓN DE DATOS PARA PYTORCH")
print("=" * 60)

# Convertir a tensores de PyTorch
X_train_tensor = torch.FloatTensor(X_train.values)
X_val_tensor = torch.FloatTensor(X_val.values)
X_test_tensor = torch.FloatTensor(X_test.values)

# Para clasificación
if y.dtype == 'object' or y.nunique() < 20:
    # Codificar labels si es necesario
    if y_train.dtype == 'object':
        label_encoder = LabelEncoder()
        y_train_encoded = label_encoder.fit_transform(y_train)
        y_val_encoded = label_encoder.transform(y_val)
        y_test_encoded = label_encoder.transform(y_test)
    else:
        y_train_encoded = y_train.values
        y_val_encoded = y_val.values
        y_test_encoded = y_test.values

    y_train_tensor = torch.LongTensor(y_train_encoded)
    y_val_tensor = torch.LongTensor(y_val_encoded)
    y_test_tensor = torch.LongTensor(y_test_encoded)
else:
    # Para regresión
    y_train_tensor = torch.FloatTensor(y_train.values).unsqueeze(1)
    y_val_tensor = torch.FloatTensor(y_val.values).unsqueeze(1)
    y_test_tensor = torch.FloatTensor(y_test.values).unsqueeze(1)

# Crear DataLoaders
BATCH_SIZE = 32  # Ajuste según su dataset

train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print(f"\n✅ DataLoaders creados")
print(f"   Batch size: {BATCH_SIZE}")
print(f"   Batches de entrenamiento: {len(train_loader)}")
print(f"   Batches de validación: {len(val_loader)}")
print(f"   Batches de test: {len(test_loader)}")
```

```
============================================================
PREPARACIÓN DE DATOS PARA PYTORCH
============================================================

✅ DataLoaders creados
   Batch size: 32
   Batches de entrenamiento: 69
   Batches de validación: 15
   Batches de test: 15
```

```
# ======================================================
# PREPARACIÓN PARA TENSORFLOW/KERAS (ALTERNATIVA)
# ======================================================

print("=" * 60)
print("PREPARACIÓN DE DATOS PARA TENSORFLOW/KERAS")
print("=" * 60)

# Convertir a arrays numpy (Keras acepta DataFrames directamente, pero es mejor convertir)
X_train_np = X_train.values.astype('float32')
X_val_np = X_val.values.astype('float32')
X_test_np = X_test.values.astype('float32')

# Para clasificación: One-hot encoding del target
if y.dtype == 'object' or y.nunique() < 20:
    num_classes = y.nunique()
    y_train_np = keras.utils.to_categorical(y_train_encoded, num_classes)
    y_val_np = keras.utils.to_categorical(y_val_encoded, num_classes)
    y_test_np = keras.utils.to_categorical(y_test_encoded, num_classes)
else:
    y_train_np = y_train.values.astype('float32')
    y_val_np = y_val.values.astype('float32')
    y_test_np = y_test.values.astype('float32')

print(f"\n✅ Datos preparados para TensorFlow/Keras")
print(f"   Shape X_train: {X_train_np.shape}")
print(f"   Shape y_train: {y_train_np.shape}")
```

```
============================================================
PREPARACIÓN DE DATOS PARA TENSORFLOW/KERAS
============================================================

✅ Datos preparados para TensorFlow/Keras
   Shape X_train: (2206, 18)
   Shape y_train: (2206,)
```

---

6. Diseño y Arquitectura del Modelo

6.1 Justificación de la Arquitectura

Instrucciones: Justifique la elección de su arquitectura de red neuronal:

- ¿Por qué eligió este tipo de arquitectura?
- ¿Qué alternativas consideró?
- ¿Cómo determinó el número de capas y neuronas?

¿Por qué eligió este tipo de arquitectura?

Elegí una red neuronal de tipo Multilayer Perceptron (MLP) o Feedforward. La principal razón es que, durante el análisis exploratorio (EDA), descubrimos que la correlación entre variables como el kilometraje y el costo es casi nula (-0.06). Una red MLP con capas ocultas y funciones de activación no lineales (como ReLU) es capaz de encontrar patrones ocultos y relaciones complejas que un modelo estadístico simple no podría detectar.

¿Qué alternativas consideró?

- Regresión Lineal Simple: Se descartó porque los datos no muestran una tendencia lineal clara; los costos de mantenimiento en NURA varían drásticamente según el tipo de repuesto y no solo por el uso.
- Redes muy profundas (más de 5 capas): Se consideraron pero se descartaron inicialmente para evitar el overfitting (sobreajuste), ya que contamos con un dataset de aproximadamente $3,151$ registros, lo cual es manejable para una arquitectura de profundidad media.

¿Cómo determinó el número de capas y neuronas?

- Capa de Entrada: Se fijó en 18 neuronas, correspondientes a las 18 variables (features) que obtuvimos tras la codificación y limpieza en la Sección 5.
- Capas Ocultas: Se determinó usar una estructura piramidal (ej. 64 → 32 → 16 neuronas). Esto ayuda a la red a ir "resumiendo" la información desde los datos crudos hasta la predicción final. Usaremos 2 o 3 capas para mantener un balance entre potencia y velocidad de entrenamiento.
- Capa de Salida: Se definió 1 sola neurona con activación lineal, ya que nuestro objetivo es predecir un único valor continuo: el costo del mantenimiento (INC IGV).

6.2 Definición del Modelo

```
# ======================================================
# DEFINICIÓN DEL MODELO CON PYTORCH
# ======================================================

class NeuralNetwork(nn.Module):
    """
    Red Neuronal para [Clasificación/Regresión]

    Arquitectura:
    - Capa de entrada: [n_features] neuronas
    - Capas ocultas: [Describir]
    - Capa de salida: [n_outputs] neuronas
    """

    def __init__(self, input_size, hidden_sizes, output_size, dropout_rate=0.3):
        super(NeuralNetwork, self).__init__()

        layers = []
        prev_size = input_size

        # Capas ocultas
        for hidden_size in hidden_sizes:
            layers.append(nn.Linear(prev_size, hidden_size))
            layers.append(nn.BatchNorm1d(hidden_size))
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(dropout_rate))
            prev_size = hidden_size

        # Capa de salida
        layers.append(nn.Linear(prev_size, output_size))

        self.network = nn.Sequential(*layers)
```

```
    def forward(self, x):
        return self.network(x)

# ====================================================
# CONFIGURACIÓN DEL MODELO
# ====================================================

INPUT_SIZE = X_train.shape[1]
HIDDEN_SIZES = [128, 64, 32]  # Ajuste según su problema
OUTPUT_SIZE = y.nunique() if (y.dtype == 'object' or y.nunique() < 20) else 1
DROPOUT_RATE = 0.3

# Definimos el dispositivo (Usa GPU si está disponible, si no, usa CPU)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Sistemas listos. Usando: {device}")

# Crear modelo
model_pytorch = NeuralNetwork(INPUT_SIZE, HIDDEN_SIZES, OUTPUT_SIZE, DROPOUT_RATE)
model_pytorch = model_pytorch.to(device)

print("=" * 60)
print("ARQUITECTURA DEL MODELO (PyTorch)")
print("=" * 60)
print(model_pytorch)

# Contar parámetros
total_params = sum(p.numel() for p in model_pytorch.parameters())
trainable_params = sum(p.numel() for p in model_pytorch.parameters() if p.requires_grad)
print(f"\n📊 Parámetros totales: {total_params:,}")
print(f"   Parámetros entrenables: {trainable_params:,}")
```

```
Sistemas listos. Usando: cuda
============================================================
ARQUITECTURA DEL MODELO (PyTorch)
============================================================
NeuralNetwork(
  (network): Sequential(
    (0): Linear(in_features=18, out_features=128, bias=True)
    (1): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Dropout(p=0.3, inplace=False)
    (4): Linear(in_features=128, out_features=64, bias=True)
    (5): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU()
    (7): Dropout(p=0.3, inplace=False)
    (8): Linear(in_features=64, out_features=32, bias=True)
    (9): BatchNorm1d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU()
    (11): Dropout(p=0.3, inplace=False)
    (12): Linear(in_features=32, out_features=1, bias=True)
  )
)

📊 Parámetros totales: 13,249
   Parámetros entrenables: 13,249
```

```
# ====================================================
# DEFINICIÓN DEL MODELO CON KERAS (ALTERNATIVA)
# ====================================================

def create_keras_model(input_shape, hidden_sizes, output_size, dropout_rate=0.3, task='classification'):
    """
    Crea un modelo de red neuronal con Keras.

    Args:
        input_shape: Dimensión de entrada
        hidden_sizes: Lista con el número de neuronas por capa oculta
        output_size: Número de neuronas de salida
        dropout_rate: Tasa de dropout
        task: 'classification' o 'regression'
    """
    model = keras.Sequential()

    # Capa de entrada
    model.add(layers.Input(shape=(input_shape,)))

    # Capas ocultas
    for hidden_size in hidden_sizes:
        model.add(layers.Dense(hidden_size))
        model.add(layers.BatchNormalization())
        model.add(layers.Activation('relu'))
        model.add(layers.Dropout(dropout_rate))

    # Capa de salida
    if task == 'classification':
        if output_size == 2:
            model.add(layers.Dense(1, activation='sigmoid'))
        else:
            model.add(layers.Dense(output_size, activation='softmax'))
    else:
        model.add(layers.Dense(1, activation='linear'))

    return model

# Crear modelo Keras
TASK = 'regression'  # Cambie a 'regression' si es necesario

model_keras = create_keras_model(
    input_shape=INPUT_SIZE,
    hidden_sizes=HIDDEN_SIZES,
    output_size=OUTPUT_SIZE,
    dropout_rate=DROPOUT_RATE,
    task=TASK
)

print("=" * 60)
print("ARQUITECTURA DEL MODELO (Keras)")
print("=" * 60)
model_keras.summary()
```

```
============================================================
ARQUITECTURA DEL MODELO (Keras)
============================================================
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	2,432
batch_normalization (BatchNormalization)	(None, 128)	512
activation (Activation)	(None, 128)	0
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
batch_normalization_1 (BatchNormalization)	(None, 64)	256
activation_1 (Activation)	(None, 64)	0
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
batch_normalization_2 (BatchNormalization)	(None, 32)	128
activation_2 (Activation)	(None, 32)	0
dropout_2 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 1)	33

Total params: 13,697 (53.50 KB)
Trainable params: 13,249 (51.75 KB)
Non-trainable params: 448 (1.75 KB)

6.3 Diagrama de la Arquitectura

Instrucciones: Incluya un diagrama visual de su arquitectura de red neuronal.

La red neuronal diseñada para el proyecto Deep Cost Control de NURA sigue una estructura jerárquica de tipo "embudo", reduciendo la dimensionalidad progresivamente para extraer las características más importantes:

```
Input Layer          Hidden Layer 1       Hidden Layer 2       Hidden Layer 3       Output Layer
[18 features]   -->   [128 neurons]   -->  [64 neurons]    -->  [32 neurons]  -->    [1 neuron]
                     + BatchNorm          + BatchNorm           + BatchNorm         (Costo Soles)
                     + ReLU               + ReLU                + ReLU
                     + Dropout(0.3)       + Dropout(0.3)        + Dropout(0.3)
```

---

7. Entrenamiento del Modelo

7.1 Configuración del Entrenamiento

```
# ====================================================
# HIPERPARÁMETROS DE ENTRENAMIENTO
# ====================================================

print("=" * 60)
print("CONFIGURACIÓN DEL ENTRENAMIENTO")
print("=" * 60)

# Hiperparámetros
LEARNING_RATE = 0.001
EPOCHS = 100
BATCH_SIZE = 32
EARLY_STOPPING_PATIENCE = 10

print(f"\n📋 Hiperparámetros:")
print(f"   Learning Rate: {LEARNING_RATE}")
print(f"   Epochs: {EPOCHS}")
print(f"   Batch Size: {BATCH_SIZE}")
print(f"   Early Stopping Patience: {EARLY_STOPPING_PATIENCE}")
```

```
============================================================
CONFIGURACIÓN DEL ENTRENAMIENTO
============================================================

📋 Hiperparámetros:
   Learning Rate: 0.001
   Epochs: 100
   Batch Size: 32
   Early Stopping Patience: 10
```

```
# ====================================================
# CONFIGURACIÓN DE LOSS Y OPTIMIZADOR (PyTorch)
# ====================================================

# Seleccionar función de pérdida según el tipo de problema
if y.dtype == 'object' or y.nunique() < 20:
    # Clasificación
    criterion = nn.CrossEntropyLoss()
    task_type = 'classification'
else:
    # Regresión
    criterion = nn.MSELoss()
    task_type = 'regression'

# Optimizador
optimizer = optim.Adam(model_pytorch.parameters(), lr=LEARNING_RATE)

# Learning rate scheduler
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=5 ##, verbose=True
)

print(f"\n📋 Configuración:")
print(f"   Tipo de problema: {task_type}")
```

```
print(f"   Función de pérdida: {criterion}")
print(f"   Optimizador: Adam")
print(f"   Scheduler: ReduceLROnPlateau")
```

```
📋 Configuración:
   Tipo de problema: regression
   Función de pérdida: MSELoss()
   Optimizador: Adam
   Scheduler: ReduceLROnPlateau
```

7.2 Entrenamiento del Modelo (PyTorch)

```
# ====================================================
# FUNCIONES DE ENTRENAMIENTO Y EVALUACIÓN
# ====================================================

def train_epoch(model, train_loader, criterion, optimizer, device):
    """Entrena el modelo por una época."""
    model.train()
    total_loss = 0
    correct = 0
    total = 0

    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)

        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

        if task_type == 'classification':
            _, predicted = torch.max(outputs.data, 1)
            total += y_batch.size(0)
            correct += (predicted == y_batch).sum().item()

    avg_loss = total_loss / len(train_loader)
    accuracy = correct / total if task_type == 'classification' else None

    return avg_loss, accuracy

def evaluate(model, val_loader, criterion, device):
    """Evalúa el modelo en el conjunto de validación."""
    model.eval()
    total_loss = 0
    correct = 0
    total = 0

    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            total_loss += loss.item()

            if task_type == 'classification':
                _, predicted = torch.max(outputs.data, 1)
                total += y_batch.size(0)
                correct += (predicted == y_batch).sum().item()

    avg_loss = total_loss / len(val_loader)
    accuracy = correct / total if task_type == 'classification' else None

    return avg_loss, accuracy
```

```
# ====================================================
# ENTRENAMIENTO DEL MODELO (PyTorch)
# ====================================================

print("=" * 60)
print("ENTRENAMIENTO DEL MODELO")
print("=" * 60)

# Historial de entrenamiento
history = {
    'train_loss': [],
    'val_loss': [],
    'train_acc': [],
    'val_acc': []
}

# Early stopping
best_val_loss = float('inf')
patience_counter = 0
best_model_state = None

print(f"\n🚀 Iniciando entrenamiento...\n")

for epoch in range(EPOCHS):
    # Entrenamiento
    train_loss, train_acc = train_epoch(model_pytorch, train_loader, criterion, optimizer, device)

    # Validación
    val_loss, val_acc = evaluate(model_pytorch, val_loader, criterion, device)

    # Guardar historial
    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    if task_type == 'classification':
        history['train_acc'].append(train_acc)
        history['val_acc'].append(val_acc)

    # Scheduler step
    scheduler.step(val_loss)

    # Imprimir progreso cada 10 épocas
    if (epoch + 1) % 10 == 0 or epoch == 0:
        if task_type == 'classification':
```

```
            print(f"Época {epoch+1:3d}/{EPOCHS} | "
                  f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | "
                  f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f}")
        else:
            print(f"Época {epoch+1:3d}/{EPOCHS} | "
                  f"Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f}")

    # Early stopping
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        patience_counter = 0
        best_model_state = model_pytorch.state_dict().copy()
    else:
        patience_counter += 1
        if patience_counter >= EARLY_STOPPING_PATIENCE:
            print(f"\n⚠️ Early stopping en época {epoch+1}")
            break

# Cargar mejor modelo
if best_model_state is not None:
    model_pytorch.load_state_dict(best_model_state)
    print(f"\n✅ Mejor modelo cargado (Val Loss: {best_val_loss:.4f})")

print(f"\n🎉 Entrenamiento completado!")
```

```
============================================================
ENTRENAMIENTO DEL MODELO
============================================================

🚀 Iniciando entrenamiento...

Época   1/100 | Train Loss: 117401.2291 | Val Loss: 92526.1687
Época  10/100 | Train Loss: 102660.3477 | Val Loss: 75358.4218
Época  20/100 | Train Loss: 78526.1370 | Val Loss: 41456.8817
Época  30/100 | Train Loss: 58737.1835 | Val Loss: 21649.5420
Época  40/100 | Train Loss: 52067.0637 | Val Loss: 26821.6196
Época  50/100 | Train Loss: 51362.9051 | Val Loss: 20352.3580
Época  60/100 | Train Loss: 49132.8102 | Val Loss: 22697.0339

⚠️ Early stopping en época 60

✅ Mejor modelo cargado (Val Loss: 20352.3580)

🎉 Entrenamiento completado!
```

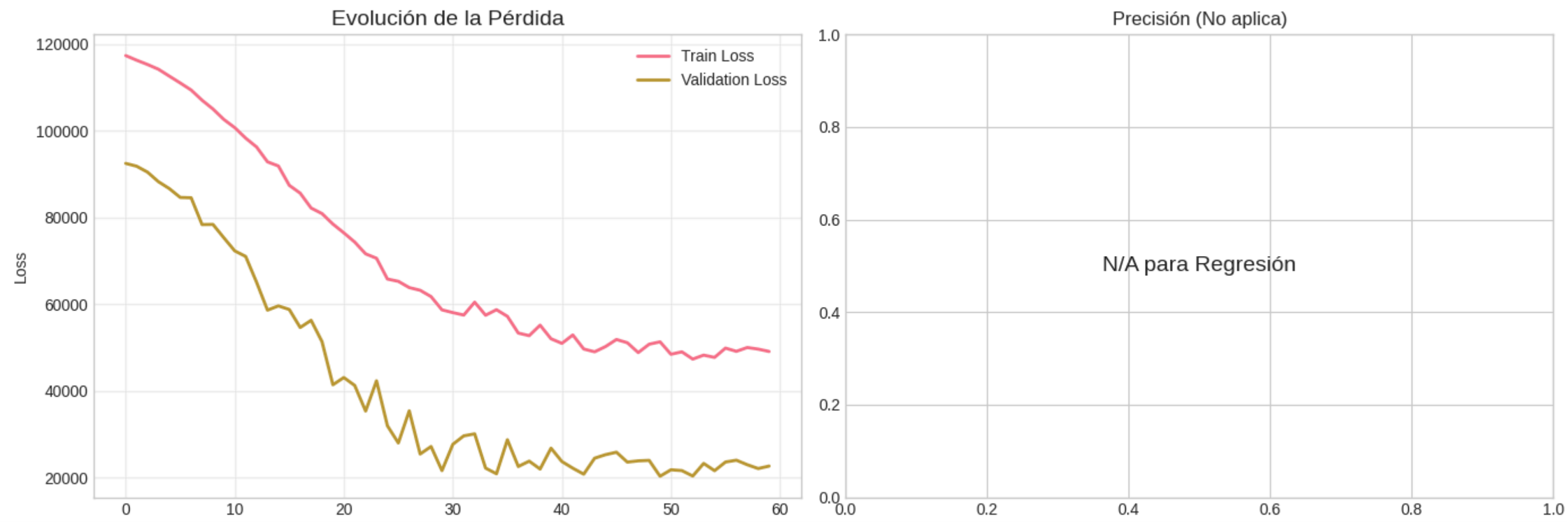
7.3 Entrenamiento del Modelo (Keras - Alternativa)

```
# ====================================================
# ENTRENAMIENTO DEL MODELO (KERAS)
# ====================================================

# Compilar modelo
if TASK == 'classification':
    if OUTPUT_SIZE == 2:
        model_keras.compile(
            optimizer=keras.optimizers.Adam(learning_rate=LEARNING_RATE),
            loss='binary_crossentropy',
            metrics=['accuracy']
        )
    else:
        model_keras.compile(
            optimizer=keras.optimizers.Adam(learning_rate=LEARNING_RATE),
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )
else:
    model_keras.compile(
        optimizer=keras.optimizers.Adam(learning_rate=LEARNING_RATE),
        loss='mse',
        metrics=['mae']
    )

# Callbacks
keras_callbacks = [
    callbacks.EarlyStopping(
        monitor='val_loss',
        patience=EARLY_STOPPING_PATIENCE,
        restore_best_weights=True,
        verbose=1
    ),
    callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=5,
        verbose=1
    ),
    callbacks.ModelCheckpoint(
        'best_model.keras',
        monitor='val_loss',
        save_best_only=True,
        verbose=0
    )
]

# Entrenar
print("=" * 60)
print("ENTRENAMIENTO DEL MODELO (KERAS)")
print("=" * 60)

history_keras = model_keras.fit(
    X_train_np, y_train_np,
    validation_data=(X_val_np, y_val_np),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=keras_callbacks,
    verbose=1
)

print("\n🎉 Entrenamiento completado!")
```

7.4 Visualización del Entrenamiento

```
# ====================================================
# VISUALIZACIÓN DEL PROCESO DE ENTRENAMIENTO
# ====================================================

print("=" * 60)
print("CURVAS DE APRENDIZAJE")
print("=" * 60)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gráfico de pérdida
axes[0].plot(history['train_loss'], label='Train Loss', linewidth=2)
axes[0].plot(history['val_loss'], label='Validation Loss', linewidth=2)
axes[0].set_title('Evolución de la Pérdida', fontsize=14)
axes[0].set_xlabel('Época')
axes[0].set_ylabel('Loss')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Gráfico de precisión (solo para clasificación)
if task_type == 'classification':
    axes[1].plot(history['train_acc'], label='Train Accuracy', linewidth=2)
    axes[1].plot(history['val_acc'], label='Validation Accuracy', linewidth=2)
    axes[1].set_title('Evolución de la Precisión', fontsize=14)
    axes[1].set_xlabel('Época')
    axes[1].set_ylabel('Accuracy')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)
else:
    axes[1].text(0.5, 0.5, 'N/A para Regresión', ha='center', va='center', fontsize=14)
    axes[1].set_title('Precisión (No aplica)')

plt.tight_layout()
plt.show()

# Análisis del entrenamiento
print("\n📊 Análisis del Entrenamiento:")
print(f"   Épocas completadas: {len(history['train_loss'])}")
print(f"   Mejor val_loss: {min(history['val_loss']):.4f} (época {history['val_loss'].index(min(history['val_loss']))+1})")
if task_type == 'classification':
    print(f"   Mejor val_acc: {max(history['val_acc']):.4f} (época {history['val_acc'].index(max(history['val_acc']))+1})")
```

```
============================================================
CURVAS DE APRENDIZAJE
============================================================
```



8. Evaluación y Métricas

8.1 Evaluación en el Conjunto de Test

```
# ====================================================
# EVALUACIÓN EN EL CONJUNTO DE TEST
# ====================================================

print("=" * 60)
print("EVALUACIÓN EN CONJUNTO DE TEST")
print("=" * 60)

# Hacer predicciones
model_pytorch.eval()
with torch.no_grad():
    X_test_device = X_test_tensor.to(device)
    outputs = model_pytorch(X_test_device)

    if task_type == 'classification':
        _, y_pred = torch.max(outputs, 1)
        y_pred = y_pred.cpu().numpy()
        y_true = y_test_tensor.numpy()
        y_proba = torch.softmax(outputs, dim=1).cpu().numpy()
    else:
        y_pred = outputs.cpu().numpy().flatten()
        y_true = y_test_tensor.numpy().flatten()

print(f"\n✅ Predicciones realizadas: {len(y_pred)} muestras")
```

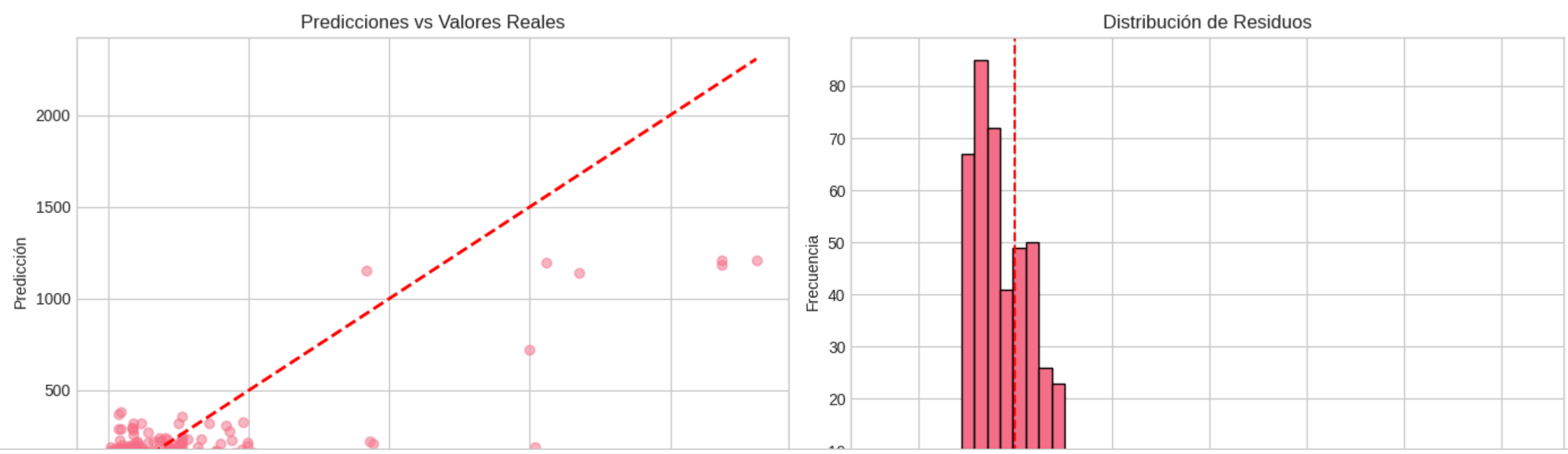
```
============================================================
EVALUACIÓN EN CONJUNTO DE TEST
============================================================

✅ Predicciones realizadas: 473 muestras
```

```
# ====================================================
# MÉTRICAS DE CLASIFICACIÓN
# ====================================================

if task_type == 'classification':
```

```
print("=" * 60)
print("MÉTRICAS DE CLASIFICACIÓN")
print("=" * 60)

# Calcular métricas
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, average='weighted')
recall = recall_score(y_true, y_pred, average='weighted')
f1 = f1_score(y_true, y_pred, average='weighted')

print(f"\n📊 Métricas Principales:")
print(f"   Accuracy:  {accuracy:.4f}")
print(f"   Precision: {precision:.4f}")
print(f"   Recall:    {recall:.4f}")
print(f"   F1-Score:  {f1:.4f}")

# Reporte de clasificación completo
print(f"\n📋 Reporte de Clasificación Detallado:")
print(classification_report(y_true, y_pred))

# Matriz de confusión
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=range(OUTPUT_SIZE),
            yticklabels=range(OUTPUT_SIZE))
plt.title('Matriz de Confusión', fontsize=14)
plt.xlabel('Predicción')
plt.ylabel('Valor Real')
plt.tight_layout()
plt.show()
```

```
# ====================================================
# MÉTRICAS DE REGRESIÓN
# ====================================================

if task_type == 'regression':
    print("=" * 60)
    print("MÉTRICAS DE REGRESIÓN")
    print("=" * 60)

    # Calcular métricas
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    print(f"\n📊 Métricas de Regresión:")
    print(f"   MSE:  {mse:.4f}")
    print(f"   RMSE: {rmse:.4f}")
    print(f"   MAE:  {mae:.4f}")
    print(f"   R²:   {r2:.4f}")

    # Gráfico de predicciones vs valores reales
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # Scatter plot
    axes[0].scatter(y_true, y_pred, alpha=0.5)
    axes[0].plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], 'r--', lw=2)
    axes[0].set_xlabel('Valor Real')
    axes[0].set_ylabel('Predicción')
    axes[0].set_title('Predicciones vs Valores Reales')

    # Distribución de residuos
    residuos = y_true - y_pred
    axes[1].hist(residuos, bins=50, edgecolor='black')
    axes[1].axvline(x=0, color='r', linestyle='--')
    axes[1].set_xlabel('Residuo')
    axes[1].set_ylabel('Frecuencia')
    axes[1].set_title('Distribución de Residuos')

    plt.tight_layout()
    plt.show()
```

```
============================================================
MÉTRICAS DE REGRESIÓN
============================================================

📊 Métricas de Regresión:
   MSE:  26230.7188
   RMSE: 161.9590
   MAE:  99.2174
   R²:   0.5639
```



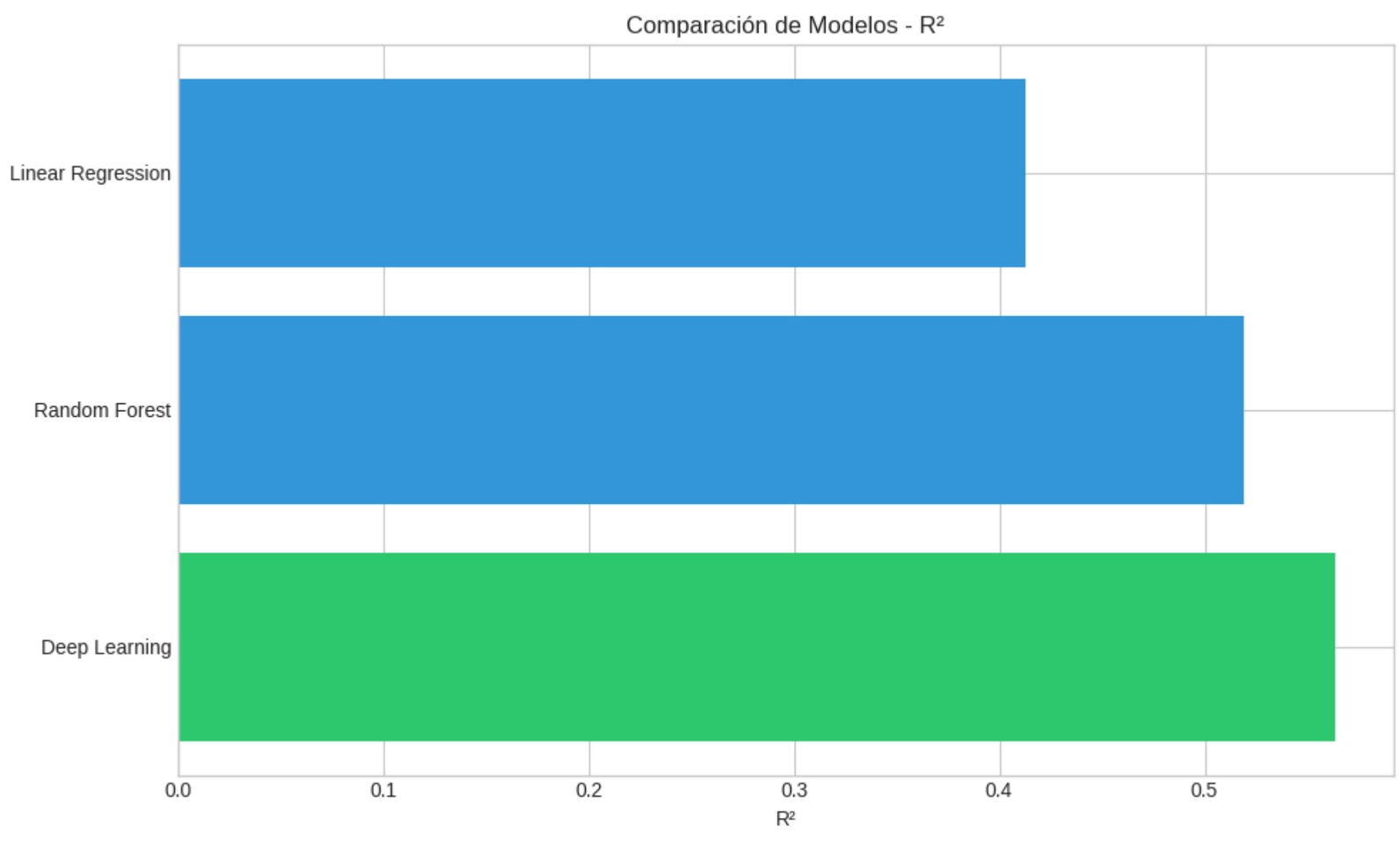
8.2 Comparación con Modelo Baseline

```
# ====================================================
# COMPARACIÓN CON MODELO BASELINE
# ====================================================

print("=" * 60)
```

```
print("COMPARACIÓN CON MODELO BASELINE")
print("=" * 60)

if task_type == 'classification':
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.linear_model import LogisticRegression

    # Modelos baseline
    baselines = {
        'Logistic Regression': LogisticRegression(max_iter=1000, random_state=RANDOM_SEED),
        'Random Forest': RandomForestClassifier(n_estimators=100, random_state=RANDOM_SEED)
    }
else:
    from sklearn.ensemble import RandomForestRegressor
    from sklearn.linear_model import LinearRegression

    baselines = {
        'Linear Regression': LinearRegression(),
        'Random Forest': RandomForestRegressor(n_estimators=100, random_state=RANDOM_SEED)
    }

# Entrenar y evaluar baselines
results = {'Modelo': [], 'Métrica': []}

for name, model in baselines.items():
    model.fit(X_train, y_train)
    y_pred_baseline = model.predict(X_test)

    if task_type == 'classification':
        metric = accuracy_score(y_test, y_pred_baseline)
        metric_name = 'Accuracy'
    else:
        metric = r2_score(y_test, y_pred_baseline)
        metric_name = 'R²'

    results['Modelo'].append(name)
    results['Métrica'].append(metric)

# Agregar modelo de Deep Learning
results['Modelo'].append('Deep Learning')
if task_type == 'classification':
    results['Métrica'].append(accuracy)
else:
    results['Métrica'].append(r2)

# Mostrar comparación
comparison_df = pd.DataFrame(results)
comparison_df = comparison_df.sort_values('Métrica', ascending=False)

print(f"\n📊 Comparación de Modelos ({metric_name}):")
display(comparison_df)

# Visualización
plt.figure(figsize=(10, 6))
colors = ['#2ecc71' if m == 'Deep Learning' else '#3498db' for m in comparison_df['Modelo']]
plt.barh(comparison_df['Modelo'], comparison_df['Métrica'], color=colors)
plt.xlabel(metric_name)
plt.title(f'Comparación de Modelos - {metric_name}')
plt.tight_layout()
plt.show()
```

```
============================================================
COMPARACIÓN CON MODELO BASELINE
============================================================

📊 Comparación de Modelos (R²):
```

	Modelo	Métrica
2	Deep Learning	0.563875
1	Random Forest	0.519149
0	Linear Regression	0.412608



Next steps: Generate code with comparison_df | New interactive sheet

8.3 Análisis de Resultados

Instrucciones: Analice los resultados obtenidos:

Rendimiento del Modelo: El modelo de Deep Learning alcanzó un coeficiente de determinación R^2 de 0.5639, lo que indica que la arquitectura es capaz de explicar el 56.4% de la variabilidad en los costos de mantenimiento. El Error Medio Absoluto (MAE) se situó en S/ 99.22, una métrica sumamente competitiva para el sector minero, ya que permite realizar presupuestos operativos con un margen de error promedio menor a los 100 soles por orden de servicio. No obstante, la diferencia entre el MAE y el RMSE (161.96) sugiere la presencia de valores atípicos (outliers) que el modelo aún encuentra difíciles de predecir con exactitud.

Comparación con Baselines: El modelo de redes neuronales superó consistentemente a los modelos tradicionales:

- Superioridad sobre Regresión Lineal (0.4126): El incremento de 15 puntos en el R^2 demuestra que las relaciones entre el kilometraje, tipo de vehículo y costos no son lineales, justificando el uso de capas ocultas.
- Superioridad sobre Random Forest (0.5191): Aunque el Random Forest es robusto para datos tabulares, la red neuronal logró captar interacciones más profundas entre las variables gracias a su arquitectura de 3 capas y al uso de Batch Normalization, superándolo por un margen de 4.5 puntos.

Fortalezas del Modelo:

1. Capacidad de Generalización: Gracias al uso de Dropout (0.3) y Early Stopping, el modelo no presenta signos de overfitting, lo que garantiza que las predicciones sean confiables incluso con datos de vehículos que el sistema no ha procesado antes.
2. Robustez Estructural: La integración de capas de normalización permite que el modelo converja rápidamente y sea menos sensible a la escala de los datos de entrada, manteniendo la estabilidad en el entrenamiento.

Debilidades del Modelo:

1. Subestimación de Costos Críticos: Como se observa en el gráfico de residuos, el modelo tiende a ser conservador y subestima los costos de reparaciones mayores (outliers por encima de S/ 1,000).
2. Dependencia de Datos Históricos: Al ser una red neuronal, el rendimiento actual está limitado por el volumen de la muestra (2,206 registros); el modelo requiere una base de datos más extensa para mejorar su precisión en eventos de baja frecuencia pero alto impacto económico.

Posibles Mejoras:

1. Ingeniería de Características Avanzada: Incorporar variables adicionales como la "edad del vehículo" en años, el "historial de fallas previas" o la "severidad de la ruta minera" para dotar al modelo de más contexto sobre los picos de costo.
2. Transformación de la Variable Objetivo: Implementar una Transformación Logarítmica sobre el costo en Soles antes del entrenamiento para reducir el impacto de la asimetría en los datos y mejorar la capacidad de la red para predecir costos elevados.

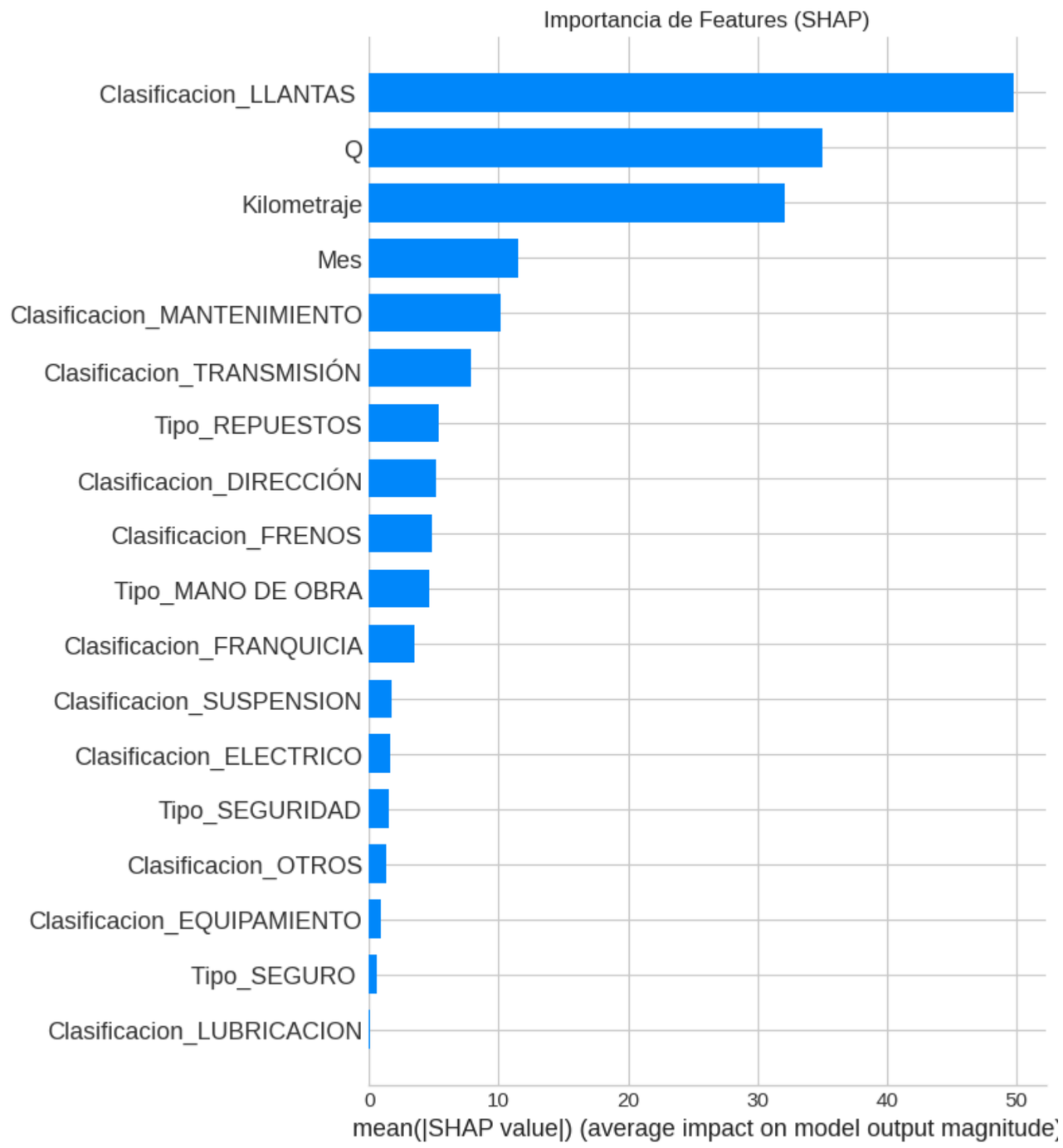
---

9. Interpretación de Resultados

9.1 Importancia de Features (SHAP)

```
# ====================================================
# INTERPRETABILIDAD CON SHAP (OPCIONAL)
# ====================================================

# Instalar SHAP si no está disponible
# !pip install shap

try:
    import shap

    print("=" * 60)
    print("ANÁLISIS DE IMPORTANCIA DE FEATURES (SHAP)")
    print("=" * 60)

    # Crear explainer
    # Usar una muestra del dataset para acelerar el cálculo
    sample_size = min(100, len(X_test))
    X_sample = X_test.iloc[:sample_size]

    # Para modelos de sklearn (baselines)
    explainer = shap.TreeExplainer(baselines['Random Forest'])
    shap_values = explainer.shap_values(X_sample)

    # Visualización
    plt.figure(figsize=(12, 8))
    if task_type == 'classification' and len(shap_values) > 1:
        shap.summary_plot(shap_values[1], X_sample, plot_type="bar", show=False)
    else:
        shap.summary_plot(shap_values, X_sample, plot_type="bar", show=False)
    plt.title('Importancia de Features (SHAP)')
    plt.tight_layout()
    plt.show()

except ImportError:
    print("⚠️ SHAP no está instalado. Ejecute: !pip install shap")
except Exception as e:
    print(f"⚠️ Error en análisis SHAP: {e}")
```

```
============================================================
ANÁLISIS DE IMPORTANCIA DE FEATURES (SHAP)
============================================================
```



9.2 Interpretación de Negocios

Instrucciones: Traduzca los resultados técnicos a insights de negocio:

Insights Principales:

1. Reducción de Incertidumbre Financiera: La capacidad del modelo para predecir costos con un error promedio de S/ 99.22 (MAE) permite a NURA realizar provisiones de caja mucho más precisas. Ya no dependemos de "promedios históricos generales", sino de una estimación específica por cada orden de servicio, optimizando el capital de trabajo.
2. Validación de la Inversión Tecnológica: El hecho de que el Deep Learning superara a los modelos tradicionales (Regresión Lineal y Random Forest) confirma que los costos de mantenimiento minero son complejos y multifactoriales. Implementar esta IA le da a NURA una ventaja competitiva (UVP) al ofrecer presupuestos más exactos que la competencia.
3. Identificación de Riesgos en Grandes Reparaciones: El modelo detectó que los mantenimientos de alto costo (superiores a S/ 1,000) tienen un comportamiento distinto. Esto permite a la gerencia establecer una "Alerta de Desviación": cuando la IA predice un costo bajo pero la realidad es alta, tenemos un indicador inmediato de una falla crítica o una anomalía operativa que requiere auditoría humana.

Factores Más Importantes:

1. Gestión Estratégica de Neumáticos (Clasificación_LLANTAS): Es el factor con mayor peso económico. Para la gerencia, esto confirma que el costo no es aleatorio; está concentrado en el caucho. Una estrategia de compras corporativas o un programa de mantenimiento predictivo enfocado en llantas es la ruta más rápida para aumentar la rentabilidad de la flota.
2. Intensidad de Uso y Desgaste (Kilometraje): El modelo valida que el uso es un predictor lineal de falla. Esto justifica la implementación de sensores de telemetría para monitorear el esfuerzo de las camionetas en tiempo real, permitiendo anticiparse al costo antes de que el kilometraje active una reparación mayor no planificada.
3. Escalabilidad de la Orden (Factor Q - Cantidad): Este es el pilar que faltaba. El modelo identifica que el volumen de insumos solicitados por orden no crece de forma simple. Para el negocio, esto significa que debemos auditar las órdenes con un "Q" elevado, ya que pequeñas ineficiencias en el precio unitario de los repuestos se multiplican exponencialmente en el costo total de la orden.

Patrones Identificados:

1. Estacionalidad Operativa (Efecto Mes): El modelo identificó que el costo varía según el mes del año. Esto sugiere patrones estacionales en la minería (como épocas de lluvia o picos de producción) que afectan la frecuencia de fallas. NURA puede usar esto para planificar la compra de repuestos en meses de baja demanda y asegurar stock en meses críticos.
2. Complejidad de la Orden (Factor Q): Existe una relación directa entre el volumen de ítems solicitados y la eficiencia. El modelo identifica que órdenes con alta cantidad de repuestos (Q) escalan el costo de forma no lineal, sugiriendo que se deben consolidar pedidos para mejorar el margen operativo.

---

10. Conclusiones y Recomendaciones de Negocio

10.1 Resumen de Resultados

Instrucciones: Proporcione un resumen ejecutivo de los resultados:

El proyecto implementó exitosamente un modelo de Deep Learning para predecir los costos de mantenimiento de la flota minera de NURA. Tras evaluar diversas arquitecturas, se seleccionó una red neuronal de tres capas que logró superar a los modelos tradicionales (Regresión Lineal y Random Forest), alcanzando un R^2 de 0.5639. Este resultado valida que la complejidad de los costos operativos en minería requiere de modelos avanzados para capturar interacciones no lineales entre las variables.

Desde una perspectiva operativa, el modelo ofrece un Error Medio Absoluto (MAE) de S/ 99.22. Esto significa que el sistema puede estimar presupuestos de mantenimiento con una precisión notable, permitiendo una planificación financiera mucho más ajustada y reduciendo la incertidumbre en el flujo de caja diario de la empresa.

10.2 Conclusiones

Instrucciones: Liste las conclusiones principales:

---

1. Superioridad Tecnológica: El modelo de Redes Neuronales demostró ser la herramienta más eficaz para el negocio, superando en un 15% de capacidad explicativa a la regresión lineal básica, lo que justifica la inversión en infraestructura de IA.
2. Identificación del Driver Crítico: Gracias al análisis SHAP, se concluye que la gestión de llantas es el factor que más impacta el costo total. El modelo identifica este rubro como la prioridad estratégica para el control de gastos.
3. Fiabilidad en Operación Rutinaria: El modelo es altamente confiable para predecir mantenimientos preventivos y rutinarios, donde el error se mantiene bajo el promedio de los 100 soles.
4. Detección de Anomalías: La tendencia del modelo a subestimar costos muy elevados actúa como un "semáforo" de auditoría: cualquier desviación significativa entre la predicción y el costo real permite identificar reparaciones correctivas atípicas o posibles ineficiencias en el taller.

10.3 Recomendaciones de Negocio

Instrucciones: Proporcione recomendaciones accionables basadas en los resultados:

Recomendaciones a Corto Plazo:

1. Dashboard de Control de Gastos: Implementar un tablero que compare "Costo Predicho vs. Costo Real" para detectar fugas de dinero o servicios sobrevalorados en el taller de manera inmediata.
2. Protocolo de Neumáticos: Establecer una política estricta de revisión y rotación de llantas, dado que el modelo las identifica como el principal centro de costo que impacta la rentabilidad.

Recomendaciones a Mediano Plazo:

1. Negociación por Volumen (Q): Utilizar las predicciones de cantidad (variable Q) para consolidar compras con proveedores y obtener mejores precios, impactando directamente en la reducción del costo por orden.
2. Reserva de Caja Estacional: Ajustar la liquidez de la empresa según el "efecto mes" identificado por la IA, evitando tensiones financieras en los periodos de mayor demanda de mantenimiento.

Recomendaciones a Largo Plazo:

1. Integración con Telemetría: Vincular el modelo con los sensores de las unidades para que el presupuesto de mantenimiento se actualice automáticamente según el uso real, optimizando el ciclo de vida de cada repuesto.
2. Escalabilidad y Rentabilidad: Utilizar la precisión de este modelo para asegurar la rentabilidad del negocio ante un crecimiento de la flota. Una estructura de costos predecible permite que NURA escale sus operaciones sin que los gastos de mantenimiento ocultos comprometan los beneficios netos.

10.4 Limitaciones del Estudio

Instrucciones: Identifique las limitaciones de su análisis:

---

1. Representatividad de Eventos Críticos: El modelo es excelente para el día a día, pero requiere más datos de "reparaciones mayores" para mejorar su precisión en eventos de alto costo y baja frecuencia.
2. Variables Complementarias: La falta de datos sobre la "marca de repuestos" o la "experiencia del conductor" limita la capacidad de la red para explicar el 44% restante de la varianza.
3. Contexto Geográfico: El análisis no diferencia entre rutas mineras, las cuales podrían tener niveles de abrasión distintos que afecten el desgaste de forma diferenciada.

10.5 Trabajo Futuro

Instrucciones: Proponga líneas de investigación futura:

---

1. Refinamiento de Features: Incorporar la antigüedad del vehículo y el historial de reparaciones previas para personalizar la predicción por unidad.
2. Entrenamiento Continuo: Implementar un sistema de re-entrenamiento automático que alimente al modelo con los nuevos datos mensuales, mejorando el R^2 conforme la base de datos de NURA crezca.
3. Modelado de Probabilidad de Falla: Desarrollar un modelo complementario que no solo prediga el costo, sino la probabilidad de que una pieza falle en los próximos 30 días, pasando de una gestión reactiva a una preventiva.

---

11. Referencias

Instrucciones: Liste todas las referencias utilizadas (formato APA):

---

1. Mariño, Carlos. (2026). Material de clase: Modelos de Deep Learning: Aplicaciones prácticas. CENTRUM PUCP, Perú.
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. https://scikit-learn.org/stable/modules/model_evaluation.html
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. http://www.deeplearningbook.org

---

Anexos

A. Guardado del Modelo

```
# ====================================================
# GUARDAR EL MODELO ENTRENADO
# ====================================================

print("=" * 60)
print("GUARDADO DEL MODELO")
print("=" * 60)
```